

Autonomous Multi-Agent Algorithm Evolution for the Capacitated Vehicle Routing Problem with Time Windows

Nuwan Ganganath
manganathan@gmail.com

ABSTRACT

We apply MengerFlock, a hierarchical multi-agent system for autonomous algorithm improvement, to evolve Hybrid Genetic Search for the Capacitated Vehicle Routing Problem with Time Windows (CVRPTW). Across two successive experiments comprising 69 autonomous trials over 14 hours, the system identifies 7 complementary improvements that reduce the competition metric ($1000 \times NV + TD$) by up to 16% on individual instances, with 66 vehicle reductions across the full 104-instance benchmark set. The net metric improvement is $-60,577$, with only 5 minor regressions (all below $+0.3\%$). The most impactful finding is a route count penalty that aligns internal search pressure with the competition objective, discovered by an unconstrained wildcard agent.

CCS CONCEPTS

• **Theory of computation** → **Design and analysis of algorithms**; • **Computing methodologies** → **Artificial intelligence**.

KEYWORDS

CVRPTW, hybrid genetic search, multi-agent systems, autonomous algorithm improvement, population management

ACM Reference Format:

Nuwan Ganganath. 2026. Autonomous Multi-Agent Algorithm Evolution for the Capacitated Vehicle Routing Problem with Time Windows. In *Genetic and Evolutionary Computation Conference (GECCO Companion '26)*, July 13–17, 2026, San Jose, Costa Rica. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3795101.3815556>

1 INTRODUCTION

The Capacitated Vehicle Routing Problem with Time Windows (CVRPTW) is a classical combinatorial optimization problem. Given a depot, a fleet of vehicles with capacity Q , and n customers each with demand q_i , time window $[e_i, l_i]$, and service duration s_i , the objective is to find routes serving all customers while minimizing $1000 \times NV + TD$.

Hybrid Genetic Search (HGS) [2] is among the most effective metaheuristics for vehicle routing, combining order crossover (OX) and selective route exchange crossover (SREX) with local search operators (relocate, swap, 2-opt, and SWAP*) in a steady-state genetic framework with dual feasible/infeasible subpopulations. HGS ranked 1st in the DIMACS 2021 VRPTW Challenge and has been extended with time-warp-based constraint handling [3]. We apply

MengerFlock [4], an autonomous multi-agent system, to systematically search for improvements in HGS-VRPTW without human intervention during the search phase.

2 SYSTEM OVERVIEW

MengerFlock [4] coordinates AI coding agents in a hierarchical architecture. A **strategist** researches the problem domain via web search, reads the seed codebase and reference papers, decomposes the code into modules, and creates researcher assignments. It composes successful changes via a regression-gated protocol and redirects agents away from exhausted directions. **Researchers** ($N=3$) each evolve an assigned module in isolated git worktrees, running an autonomous loop: hypothesize, implement, commit, build, evaluate, keep or discard. Each hypothesis starts from a fresh branch off the main branch to prevent measurement drift. A **wildcard** agent works unconstrained from the original unmodified seed with no web search and no visibility into other agents' results, forcing genuine novelty.

Agents communicate through shared files on the filesystem; git provides state management. Each candidate must pass a zero-regression gate on representative instances covering all families (C1, C2, R1, R2, RC1, RC2) and sizes (100, 200, 400 clients) before being composed onto the main branch. Composition follows a single-keep protocol: each keep is cherry-picked immediately onto main, tested against the gate, and reverted if any gate instance regresses. Instances are split into train (60%), validation (20%), and holdout (20%) sets, stratified by family.

3 DISCOVERED IMPROVEMENTS

Two successive experiments comprising 69 trials were conducted. Experiment 1 (8 hours, 57 trials, 4 agents) evolved HGS-VRPTW [3] and merged 5 improvements. Experiment 2 (6 hours, 12 trials, 4 agents) started from the evolved seed of experiment 1 and yielded 2 further improvements.

Experiment 1 yielded 5 improvements focused on population dynamics and local search:

(1) **Tournament size 2→5**. Parent selection tournament increased from binary to size 5, drawing from the full combined feasible/infeasible pool. The response on R1_4_5 (400 clients) is non-monotonic: sizes 3 (+12), 4 (+22), 7 (+211) all regress, while size 5 uniquely improves by -924 . This suggests a phase transition in the balance between selection pressure and population diversity. Restricting to feasible-only tournaments regresses R-type instances by $+374$, because infeasible solutions contain structural innovations (fewer vehicles with slight time window violations) that lead to feasible improvements after repair.

(2) **Biased fitness optimization**. Biased fitness recomputation moved from per-tournament to per-parent-selection, eliminating 4–20× redundant computation per offspring. This is a pure

This work is licensed under a Creative Commons Attribution 4.0 International License. *GECCO Companion '26, July 13–17, 2026, San Jose, Costa Rica*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2488-6/2026/07.
<https://doi.org/10.1145/3795101.3815556>

speedup, yielding -865 further improvement on R1_4_5 through additional GA iterations within the 120s budget.

(3) **Diversity gate.** After computing broken-pairs distance from a new offspring to all existing population members, reject the offspring if its minimum distance is below 0.02 and the subpopulation is at minimum size. This prevents population collapse under tournament-5 pressure: without it, near-identical high-fitness individuals dominate both subpopulations, reducing crossover effectiveness. Threshold 0.02 is uniquely optimal: 0.01 yields +24 (too lenient), 0.05 yields +45 (too strict). This modification is *inert* without tournament-5, as binary tournament creates insufficient pressure for collapse. Together, tournament-5 and the diversity gate produce -1040 on R1_4_5, while neither improves it alone.

(4) **Dynamic parameter adaptation.** Enabled existing but disabled instance-adaptive configuration: 100% SWAP* intensification for small-route instances, population growth after stagnation for large instances, and adaptive granular neighborhood sizing.

(5) **Unconditional SWAP*.** Removed short-circuit that skipped time-window-aware SWAP* when distance-based SWAP* found an improvement. Distance-based SWAP* evaluates by Euclidean distance; time-window-aware SWAP* evaluates by time window compatibility using cached $O(1)$ move evaluation. These find complementary improving moves and should both execute. Discovered after 6 discarded hypotheses (or-opt-3, circle sector tolerance, TW reordering, intensification probability).

Experiment 2 yielded 2 improvements focused on objective function shaping:

(6) **Route count penalty.** Added a penalty of 200 per route to the penalized cost function. HGS-VRPTW penalizes capacity and time window violations but not vehicle count. Since the competition metric weights each vehicle at 1000, biasing the search toward fewer vehicles provides large metric gains. The value 200 (not 1000) provides sufficient pressure without sacrificing travel distance. This was the most impactful change, accounting for most of the $-58,439$ incremental gain over experiment 1, discovered by the wildcard agent.

(7) **Penalty recalculation fix.** Fixed the penalty management procedure to account for the route penalty and wait time when adjusting penalty coefficients. Without this, the coefficients oscillate because the route penalty distorts the feasibility signal.

4 RESULTS

Both baseline (unmodified HGS-VRPTW) and evolved solvers were evaluated on all 104 competition instances (56 Solomon 100-client [1], 48 Gehring-Homberger 200/400-client [5]) at 120s, seed 42.

Table 1: Top 5 improved instances (of 56 improved, out of 104 total).

Instance	Fam.	Baseline	Evolved	$\Delta\%$	ΔNV
r202	R2	18296	15369	-16.00	-3
rc206	RC2	17511	14786	-15.56	-3
r204	R2	12313	10453	-15.11	-2
rc202	RC2	18923	16157	-14.62	-3
r201	R2	19432	16803	-13.53	-3

Of 104 instances: 56 improved, 5 regressed, 43 unchanged (net $\Delta = -60,577$). A total of 66 vehicle reductions were achieved. All 5 regressions are below +0.3% and none involve vehicle count changes.

Table 2: Family-level summary across 104 instances.

Family	Inst.	Impr.	Regr.	Net Δ	ΔNV
R1	20	14	1	$-11,746$	-13
C1	17	3	0	$-1,256$	-1
R2	19	18	1	$-25,908$	-28
C2	16	4	0	$-3,742$	-4
RC1	16	5	2	$-3,295$	-4
RC2	16	12	1	$-14,630$	-16

R2-type instances (random locations, wide time windows) benefit most, with 18 of 19 improved and 28 fewer vehicles. The genetic module (11 trials) produced 0 keeps; local search dominates so strongly that crossover modifications are undetectable.

5 DISCUSSION AND CONCLUSION

Two categories of improvement emerged across the two experiments. Experiment 1 found that population dynamics (tournament selection + diversity maintenance) provide the most leverage in well-tuned metaheuristics where local search dominates solution quality. The synergy between tournament-5 and the diversity gate is notable: neither is effective alone, and the specific parameters (size exactly 5, threshold exactly 0.02) exhibit narrow optima that manual tuning would likely miss. Experiment 2 found that objective function shaping, adding a route count penalty to align internal search pressure with the competition metric, is even more impactful, accounting for most of the $-58,439$ incremental improvement over experiment 1.

The wildcard agent was disproportionately productive, discovering 6 keeps from 18 trials (33%) compared to 3 keeps from 49 directed trials (6%). Both experiment-2 improvements came from the wildcard, demonstrating the value of unconstrained exploration in mature codebases. Several literature-based hypotheses failed: PyVRP penalty parameters [6] regressed in this C++ implementation, or-opt-3 showed zero effect, and SREX-only crossover hurt R-type instances. These results demonstrate that autonomous algorithm evolution can find non-obvious improvements in highly optimized metaheuristics.

REFERENCES

- [1] M. M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper. Res.*, 35(2):254–265, 1987.
- [2] T. Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Comput. Oper. Res.*, 140:105643, 2022.
- [3] E. Kool, J. Gromicho, W. Quak, J. van Kreveld. Hybrid genetic search for the CVRP with time windows. Tech. rep., ORTEC, 2022.
- [4] N. Ganganath. MengerFlock: A hierarchical multi-agent system that evolves algorithms through autonomous experimentation, 2026. <https://github.com/manganath/mengerflock>.
- [5] H. Gehring, J. Homberger. A parallel two-phase metaheuristic for routing problems with time windows. *Asia-Pac. J. Oper. Res.*, 18(1):35–47, 2001.
- [6] N. Wouda, L. Lan, W. Kool. PyVRP: A high-performance VRP solver package. *INFORMS J. Comput.*, 36(3):615–630, 2024.